Tutorial II.II - Package Management

Optimization with Julia

Introduction

Welcome to this beginner-friendly guide on understanding packages and package management in Julia!

Think of packages as pre-written sets of tools that extend what Julia can do. It's like having a toolbox where you can add new tools (packages) to help you solve specific problems. For example, there are packages for working with data, creating visualizations, or solving complex math problems.

The best part? Most Julia packages are free to use, thanks to the open-source community!

> **i** Note
>
> While we'll practice some commands here, you'll typically manage packages in Julia's REPL (Read-Eval-Print Loop), which is like Julia's command center.

Section 1 - Using the Package Manager

Julia's built-in package manager, Pkg, provides a robust set of tools for managing packages. To utilize these tools, start by importing the Pkg module. This module allows you to add, update, and remove packages, and manage environments efficiently. Note, `import PackageName` lets you access exported functions with `PackageName.function`, while `using PackageName` imports all exported names into the local namespace for direct access.

Exercise 1.1 - Import the Pkg Module

Import the Pkg module to start managing packages effectively.

```
# YOUR CODE BELOW
```

```
# Test your answer
try
    Pkg.update()
    println("Pkg module imported successfully and packages were updated!")
catch e
    @error "The Pkg module was not imported yet! Have you used the correct
syntax?"
end
```

Section 2 - Adding Packages

Adding packages in Julia is straightforward using the `Pkg.add("PackageName")` function. Replace `PackageName` with the actual package name you wish to add.

Exercise 2.1 - Add the DataFrames Package

Let's add a popular package called DataFrames. It's great for working with structured data, like spreadsheets.

```
# YOUR CODE BELOW
```

```
# Test your answer
try
    using DataFrames
    println("Package added successfully!")
catch e
    @error "Package was not added yet! Have you used the correct syntax?"
end
```

Section 3 - Updating and Removing Packages

Just like apps on your phone, packages can be updated or removed when you no longer need them.

- To update a package: `Pkg.update("PackageName")`
- To update all packages: `Pkg.update()`
- To remove a package: `Pkg.rm("PackageName")`

If you wanted to update DataFrames, you'd use `Pkg.update("DataFrames")`. To remove it, you'd use `Pkg.rm("DataFrames")`.

Section 4 - Managing Environments

Think of environments as separate workspaces for different projects. It's like having different folders for different school subjects - each folder contains only the books and supplies you need for that subject. Using environments helps you:

- Keep your projects organized and reproducible
- Avoid conflicts between different package versions
- Share your project setup easily with others

When you create or activate an environment, Julia generates two important files:

1. `Project.toml`: Lists the packages your project directly depends on
2. `Manifest.toml`: Contains a complete record of all packages and their exact versions

These files ensure that anyone can recreate your exact setup.

4.1 Why Environments Matter

> Tip
>
> Even if you're working primarily with Jupyter notebooks, understanding environments will save you from headaches later! When you encounter a "Package not found" error, knowing how to activate the right environment is the solution.

For this course, we've prepared an environment with all the packages you'll need. This means you don't have to install each package individually - you just need to activate the course environment and you're ready to go!

4.2 Activating the Course Environment

The course materials come with a pre-configured environment containing all necessary packages. To use it, you need to activate it and then instantiate (download) the packages.

Here's how to activate an environment at a specific path:

```
import Pkg
Pkg.activate("/path/to/course/folder")
Pkg.instantiate()  # Downloads all packages listed in Project.toml
```

The `instantiate()` command reads the Project.toml file and installs all the required packages. You only need to run this once when you first set up the course.

Exercise 4.1 - Activate the Course Environment

Activate the course environment. Replace the path below with the actual path to your course folder (where you downloaded the course materials).

```
# YOUR CODE BELOW
# Hint: Use Pkg.activate("path/to/your/course/folder")
# Then use Pkg.instantiate() to install all packages
```

```
# Test your answer
try
    Pkg.status()
    println("Environment activated! Check the package list above.")
catch e
    @error "Something went wrong. Make sure you've imported Pkg first!"
end
```

4.3 Activating an Environment in Your Current Directory

Often, you'll want to create an environment right in the folder where you're working. The simplest way to do this is using a dot (`.`) which means "the current directory":

```
import Pkg
Pkg.activate(".")
```

This is particularly useful when:

- You're starting a new project in a specific folder
- You want to keep your packages local to one project
- You're working in a notebook and want packages specific to that folder

> **Note**
>
> When you activate a new environment that doesn't have a Project.toml yet, Julia will create one for you automatically when you add your first package.

Exercise 4.2 - Create and Activate a Local Environment

Create and activate an environment in your current working directory.

```
# YOUR CODE BELOW
# Hint: Use the dot notation to refer to the current directory
```

```
# Test your answer
# After running your code, this should show an empty or minimal environment
Pkg.status()
println("Local environment activated!")
```

4.4 Checking What's Installed

To see which packages are installed in your current environment, use `Pkg.status()`:

```
Pkg.status()
```

This shows you:

- The name of each installed package
- The version number
- Whether the package was added directly or as a dependency

> **Tip**
>
> If you see fewer packages than expected, you might be in the wrong environment! Use `Pkg.status()` to diagnose package issues.

Exercise 4.3 - Check Your Environment Status

Check what packages are available in your current environment.

```
# YOUR CODE BELOW
```

```
# Expected: You should see a list of packages
# If the list is empty, you may need to activate the course environment
println("Tip: If the list is empty, try activating the course environment!")
```

4.5 Adding Packages to Your Environment

Once you've activated an environment, any packages you add will be installed into that specific environment:

```
# First, make sure you're in the right environment
Pkg.activate(".")  # or your specific path

# Then add packages
Pkg.add("PackageName")
```

You can also add multiple packages at once:

```
Pkg.add(["JuMP", "HiGHS", "CSV", "Plots"])
```

> **i** Main Course Packages
>
> For this course, the main packages you'll need are:
>
> - **JuMP** and **HiGHS**: For optimization problems
> - **DataFrames** and **CSV**: For working with data
> - **Plots** and **StatsPlots**: For visualization

4.6 Keeping Your Environment Active in Notebooks

When working with Jupyter notebooks, you'll want to make sure the correct environment is active. Here's a reliable pattern to use at the start of your notebooks:

```
# Add this at the top of your notebook
import Pkg
Pkg.activate("/path/to/course/folder")  # Activate course environment

# Now you can use the packages
using JuMP, HiGHS
using DataFrames, CSV
using Plots
```

Pro Tips for Notebooks

1. **Always activate first**: Put the `Pkg.activate()` command in the first cell of your notebook
2. **Use absolute paths**: This ensures the environment is found regardless of where Jupyter starts
3. **Run activation each session**: The environment needs to be activated every time you restart the notebook kernel

Note

If you keep your notebook in the same folder as the course materials, you can use `Pkg.activate(".")` and Julia will find the Project.toml in that directory.

Conclusion

Congratulations! You've completed the tutorial on packages and package management in Julia. These skills are important for effectively managing and utilizing external libraries. Continue to the next file to learn more.

Solutions

You will find the solutions to all exercises online here in the `solutions` folders for each part. However, I strongly encourage you to work on these exercises independently without searching explicitly for the exact answers to the exercises. Understanding someone else's solution is very different from developing your own. Use the lecture notes and try to solve the exercises on your own. This approach will significantly enhance your learning and problem-solving skills.

Remember, the goal is not just to complete the exercises, but to understand the concepts and improve your programming abilities. If you encounter difficulties, review the lecture materials, experiment with different approaches, and don't hesitate to ask for clarification during class discussions.